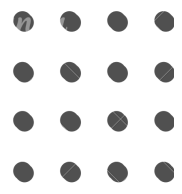




VMES Grinds

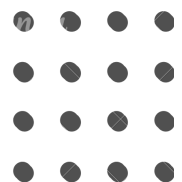




VMES Grinds

Pilot Program - Week 1 - Day 3

Arrays • Greedy • Two Pointers





Leetcode: 42 Trapping Rain Water



Moe Myint Mo San



42. Trapping Rain Water

Solved 

Hard

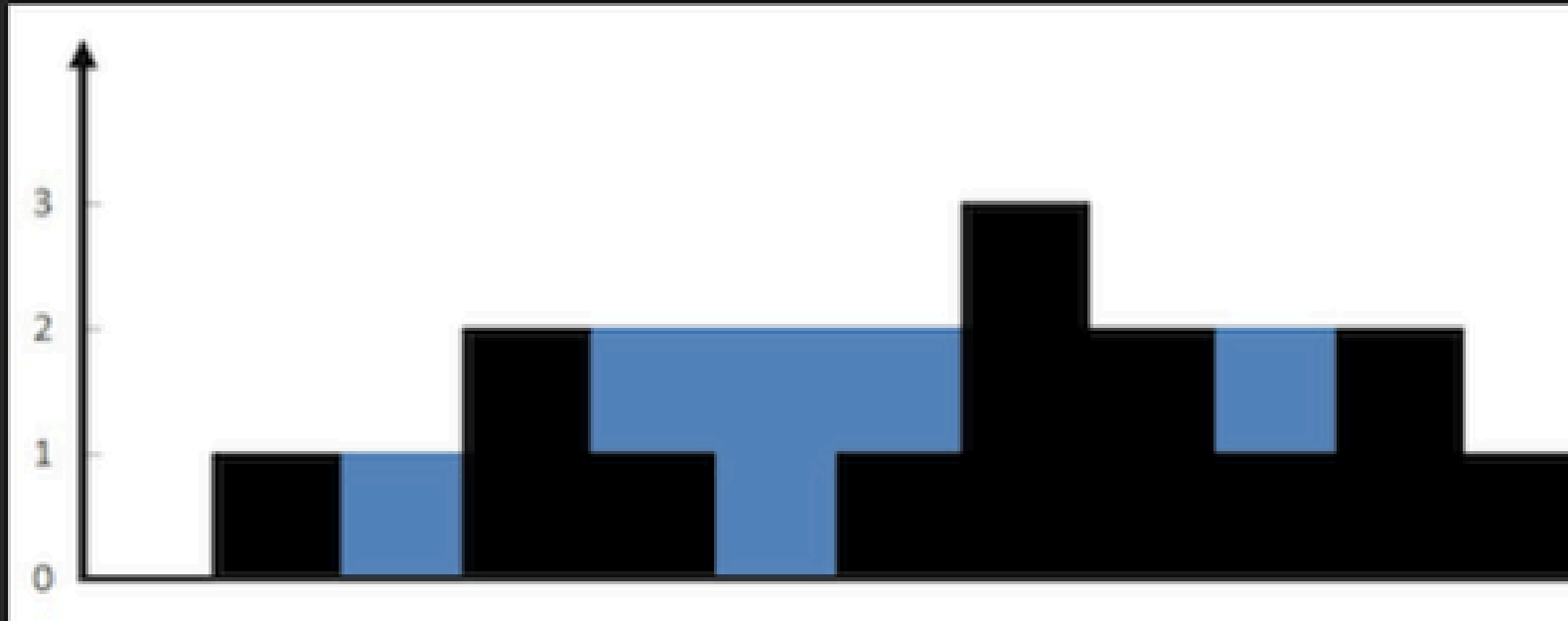
 Topics

 Companies

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.



Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9



Constraints:

- `n == height.length`
- `1 <= n <= 2 * 104`
- `0 <= height[i] <= 105`



Problem Explanation Thought Process





Initial Thought Process

- **Linear iteration through the Array**
- **Note the possible water container walls**
- **Calculate each area for every index if it's valid**



Thought Process

- Visualize bars of different heights forming walls.
- Each index i can hold water only if there are taller bars on both sides.
- The trapped amount = $\min(\text{max_left}[i], \text{max_right}[i]) - \text{height}[i]$.
- Goal $\rightarrow$ sum all trapped water units.



Algorithm Evolution Overview





Approach	Time	Space	Core Idea
Brute Force	$O(n^2)$	$O(1)$	Scan both sides for each bar
Prefix & Suffix Arrays	$O(n)$	$O(n)$	Precompute left/right maxima
Stack	$O(n)$	$O(n)$	“Valley filling” using monotonic stack
Two-Pointer (Greedy)	$O(n)$	$O(1)$	Track both sides dynamically



Brute Force (Naïve)





```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0
        n = len(height)
        res = 0

        for i in range(n):
            leftMax = rightMax = height[i]

            for j in range(i):
                leftMax = max(leftMax, height[j])
            for j in range(i + 1, n):
                rightMax = max(rightMax, height[j])

            res += min(leftMax, rightMax) - height[i]
        return res
```




Idea:

For every bar i , find highest wall on the left and right $\rightarrow$
 $\text{water}[i] = \min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$.

```
for i in range(n):  
    leftMax = rightMax = height[i]  
  
    for j in range(i):  
        leftMax = max(leftMax, height[j])  
    for j in range(i + 1, n):  
        rightMax = max(rightMax, height[j])  
  
    res += min(leftMax, rightMax) - height[i]
```

Analysis:

- Time $O(n^2)$ Memory $O(1)$
- Too slow, but builds intuition:
- "Every bar depends on both sides."



Prefix & Suffix Arrays (DP)





```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n == 0:
            return 0

        leftMax = [0] * n
        rightMax = [0] * n

        leftMax[0] = height[0]
        for i in range(1, n):
            leftMax[i] = max(leftMax[i - 1], height[i])

        rightMax[n - 1] = height[n - 1]
        for i in range(n - 2, -1, -1):
            rightMax[i] = max(rightMax[i + 1], height[i])

        res = 0
        for i in range(n):
            res += min(leftMax[i], rightMax[i]) -
height[i]return res
```




Idea: Precompute boundaries to avoid rescanning.

Steps

1. $\text{leftMax}[i] = \max(\text{height}[0..i])$
2. $\text{rightMax}[i] = \max(\text{height}[i..\text{end}])$
3. Add $\min(\text{leftMax}[i], \text{rightMax}[i]) - \text{height}[i]$ to result

```
leftMax[0] = height[0]
for i in range(1,n): leftMax[i] = max(leftMax[i-1], height[i])

rightMax[-1] = height[-1]
for i in range(n-2,-1,-1): rightMax[i] = max(rightMax[i+1],
height[i])
for i in range(n):
    res += min(leftMax[i], rightMax[i]) - height[i]
```

Analysis:

- Time $O(n)$ Memory $O(n)$
- Clear, linear time solution.

First efficient improvement; teaches boundary precomputation.



Stack (Valley Filling)





```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0
        stack = []
        res = 0

        for i in range(len(height)):
            while stack and height[i] >=
height[stack[-1]]:
                mid = height[stack.pop()]
                if stack:
                    right = height[i]
                    left = height[stack[-1]]
                    h = min(right, left) - mid
                    w = i - stack[-1] - 1
                    res += h * w
                stack.append(i)
        return res
```



Intuition:

- Maintain a monotonic decreasing stack of indices.
- When a higher bar arrives → right wall found → trap water between.

Logic

1. Push indices while height decreases.
2. On taller bar: pop mid (bottom), compute bounded water.

```
while stack and height[i] >= height[stack[-1]]:
    mid = stack.pop()
    if stack:
        h = min(height[i], height[stack[-1]]) -
height[mid]
        res += h * (i - stack[-1] - 1)
    stack.append(i)
```



Analysis:

- Time $O(n)$ Memory $O(n)$
- One-pass method; good for visualizing valleys.

Useful pattern for bounded-area and histogram problems.



Two-Pointer (Greedy Optimal)





```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        l, r = 0, len(height) - 1
        leftMax, rightMax = height[l], height[r]
        res = 0
        while l < r:
            if leftMax < rightMax:
                l += 1
                leftMax = max(leftMax, height[l])
                res += leftMax - height[l]
            else:
                r -= 1
                rightMax = max(rightMax, height[r])
                res += rightMax - height[r]
        return res
```



Key Insight:

Water on one side depends only on the smaller wall.

→ Move the pointer on that side inward and accumulate water immediately.

```
l, r = 0, len(height)-1
leftMax, rightMax = height[l], height[r]
res = 0
while l < r:
    if leftMax < rightMax:
        l += 1
        leftMax = max(leftMax, height[l])
        res += leftMax - height[l]
    else:
        r -= 1
        rightMax = max(rightMax, height[r])
        res += rightMax - height[r]
```




Why It Works

- The taller side guarantees closure for the shorter one.
- So water bounded by the shorter wall is safe to add now.

Performance

- Time $O(n)$ Memory $O(1)$
- Clean logic, no extra memory — real-world and interview favorite.



Complexity Comparison





Method	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Simple but slow
Prefix/Suffix	$O(n)$	$O(n)$	Precompute boundaries
Stack	$O(n)$	$O(n)$	Valley modeling
Two-Pointer	$O(n)$	$O(1)$	Optimal solution



Key Takeaway





- Core formula $\rightarrow \min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$.
- Prefix-Suffix $\rightarrow$ DP precomputation concept.
- Stack $\rightarrow$ Valley and boundary intuition.
- Two-Pointer $\rightarrow$ Greedy $O(1)$ space optimization.
- Pattern extends to many two-sided problems (e.g. container with most water).



Thank You !





Extra Study Material –Visualizations–





Prefix & Suffix Arrays (DP)





```
height    = [0, 1, 0, 2, 1, 0, 3]
leftMax    = [0, 1, 1, 2, 2, 2, 3]
rightMax   = [3, 3, 3, 3, 3, 3, 3]
water      = [0, 0, 1, 0, 1, 2, 0]
Total = 4
```



Stack (Valley Filling)





Step	i	height[i]	Stack (indices)	Water trapped	Explanation
0	0	0	0 [0]	0	Start
1	1	1	1 [1]	0	taller than 0, no water since stack empty after pop
2	2	2	0 [1,2]	0	descending
3	3	3	2 pop 2 → mid=0, left=1, right=2 → h=1-0=1, w=3-1-	trapped water between index 1 and 3	
4	4	4	1 [1,3,4]	0	descending again
5	5	5	0 [1,3,4,5]	0	descending
6	6	6	3 pop 5(mid=0), left=1, right=3 → h=1, w=6-4-1=1 → +1pop 4(mid=1), left=2, right=3 → h=1, w=6-3-1=2 → +2pop 3(mid=2),	total += 3	



Two-Pointer (Greedy Optimal)





Step	l	r	leftMax	rightMax	height[l] / height[r]	Action	res
0	0	6	0	3	leftMax < rightMax	move left → l=1	0
1	1	6	max(0,1)=1	3	leftMax < rightMax	move left → l=2, + (1-0)=1	1
2	2	6	max(1,0)=1	3	leftMax < rightMax	move left → l=3, + (2-2)=0	1
3	3	6	2	3	leftMax < rightMax	move left → l=4, + (2-1)=1	2
4	4	6	2	3	leftMax < rightMax	move left → l=5, + (2-0)=2	4
5	5	6	2	3	leftMax < rightMax	move left → l=6	4